PKG 04: LINKED BAG

Our bag is an abstract bag. This abstract data type (ADT) bag holds unordered and possibly duplicated objects. The objects have the same or related data types…

```
// @author Frank M. Carrano and Timothy M. Henry
public interface BagInterface<T> {

    // Same as ARRAY BAG's interface.

} // end BagInterface
```

```
public final class LinkedBag<T> implements BagInterface<T> {

    private Node firstNode;       // Reference to first node
    private int numberOfEntries;

    public LinkedBag() {
        firstNode = null;  Name firstNode as handle in assignment 03
        numberOfEntries = 0;
    } // end default constructor

    @Override
    public boolean isEmpty() {
        return numberOfEntries == 0;
    } // end isEmpty

    @Override
    public int getCurrentSize() {
        return numberOfEntries;
    } // end getCurrentSize

    @Override
    public boolean add(T newEntry) // OutOfMemoryError possible
    {
        // Add to beginning of chain:
        Node newNode = new Node(newEntry);
        newNode.next = firstNode; // Make new node reference rest of chain
        // (firstNode is null if chain is empty)
        firstNode = newNode;      // New node is at beginning of chain
        numberOfEntries++;

        return true;
    } // end add

    @Override
    public T[] toArray() {
        // The cast is safe because the new array contains null entries
        @SuppressWarnings("unchecked")
        T[] result = (T[]) new Object[numberOfEntries]; // Unchecked cast

        int index = 0;
        Node currentNode = firstNode;
        while ((index < numberOfEntries) && (currentNode != null)) {
            result[index] = currentNode.data;
            index++;
            currentNode = currentNode.next;
        } // end while

        return result;
    } // end toArray
```

```java
    @Override
    public int getFrequencyOf(T anEntry) {
        int frequency = 0;

        int counter = 0;
        Node currentNode = firstNode;
        while ((counter < numberOfEntries) && (currentNode != null)) {
            if (anEntry.equals(currentNode.data)) {
                frequency++;
            } // end if

            counter++;
            currentNode = currentNode.next;
        } // end while

        return frequency;
    } // end getFrequencyOf

    @Override
    public boolean contains(T anEntry) {
        boolean found = false;
        Node currentNode = firstNode;

        while (!found && (currentNode != null)) {
            if (anEntry.equals(currentNode.data)) {
                found = true;
            } else {
                currentNode = currentNode.next;
            }
        } // end while

        return found;
    } // end contains

    private Node getReferenceTo(T anEntry) {
        boolean found = false;
        Node currentNode = firstNode;

        while (!found && (currentNode != null)) {
            if (anEntry.equals(currentNode.data)) {
                found = true;
            } else {
                currentNode = currentNode.next;
            }
        } // end while

        return currentNode;
    } // end getReferenceTo


    @Override
    public void clear() {
        while (!isEmpty()) {
            remove();
        }
    } // end clear
```

In array bag we remove the last
In linked bag we remove the first

```java
    @Override
    public T remove() {
        T result = null;
        if (firstNode != null) {
            result = firstNode.data;
            firstNode = firstNode.next; // Remove first node from chain
            numberOfEntries--;
        } // end if

        return result;
    } // end remove

    @Override
    public boolean remove(T anEntry) {
        boolean result = false;
        Node nodeN = getReferenceTo(anEntry);

        if (nodeN != null) {
            nodeN.data = firstNode.data; // Replace located entry with entry in first
node

            firstNode = firstNode.next;  // Remove first node
            numberOfEntries--;

            result = true;
        } // end if

        return result;
    } // end remove

//-------------------------------------------------------//
    private class Node {

        private T data; // Entry in bag
        private Node next; // Link to next node

        private Node(T dataPortion) {
            this(dataPortion, null);
        } // end constructor

        private Node(T dataPortion, Node nextNode) {
            data = dataPortion;
            next = nextNode;
        } // end constructor
    } // end Node
//-------------------------------------------------------//


} // end LinkedBag
```

```java
public class LinkedBagDemo {

    public static void main(String[] args) {
        // Tests on a bag that is empty
        System.out.println("Creating an empty bag.");
        BagInterface<String> aBag = new LinkedBag<>();
        displayBag(aBag);
        testIsEmpty(aBag, true);
        String[] testStrings1 = {"", "B"};
        testFrequency(aBag, testStrings1);
        testContains(aBag, testStrings1);
        testRemove(aBag, testStrings1);

        // Adding strings
        String[] contentsOfBag = {"A", "D", "B", "A", "C", "A", "D"};
        testAdd(aBag, contentsOfBag);

        // Tests on a bag that is not empty
        testIsEmpty(aBag, false);
        String[] testStrings2 = {"A", "B", "C", "D", "Z"};
        testFrequency(aBag, testStrings2);
        testContains(aBag, testStrings2);

        // Removing strings
        String[] testStrings3 = {"", "B", "A", "C", "Z"};
        testRemove(aBag, testStrings3);

        System.out.println("\nClearing the bag:");
        aBag.clear();
        testIsEmpty(aBag, true);
        displayBag(aBag);
    } // end main

    // Tests the method add.
    private static void testAdd(BagInterface<String> aBag, String[] content) {
        System.out.print("Adding to the bag: ");
        for (int index = 0; index < content.length; index++) {
            aBag.add(content[index]);
            System.out.print(content[index] + " ");
        } // end for
        System.out.println();

        displayBag(aBag);
    } // end testAdd

    // Tests the two remove methods.
    private static void testRemove(BagInterface<String> aBag, String[] tests) {
        for (int index = 0; index < tests.length; index++) {
            String aString = tests[index];
            if (aString.equals("") || (aString == null)) {
                // test remove()
                System.out.println("\nRemoving a string from the bag:");
                String removedString = aBag.remove();
                System.out.println("remove() returns " + removedString);
            } else {
                // test remove(aString)
                System.out.println("\nRemoving \"" + aString + "\" from the bag:");
                boolean result = aBag.remove(aString);
                System.out.println("remove(\"" + aString + "\") returns " + result);
            } // end if

            displayBag(aBag);
        } // end for
```

```
    } // end testRemove

    // Tests the method isEmpty.
    // correctResult indicates what isEmpty should return.
    private static void testIsEmpty(BagInterface<String> aBag, boolean correctResult)
{
        System.out.print("Testing isEmpty with ");
        if (correctResult) {
            System.out.println("an empty bag:");
        } else {
            System.out.println("a bag that is not empty:");
        }

        System.out.print("isEmpty finds the bag ");
        if (correctResult && aBag.isEmpty()) {
            System.out.println("empty: OK.");
        } else if (correctResult) {
            System.out.println("not empty, but it is empty: ERROR.");
        } else if (!correctResult && aBag.isEmpty()) {
            System.out.println("empty, but it is not empty: ERROR.");
        } else {
            System.out.println("not empty: OK.");
        }
        System.out.println();
    } // end testIsEmpty

    // Tests the method getFrequencyOf.
    private static void testFrequency(BagInterface<String> aBag, String[] tests) {
        System.out.println("\nTesting the method getFrequencyOf:");
        for (int index = 0; index < tests.length; index++) {
            System.out.println("In this bag, the count of " + tests[index]
                    + " is " + aBag.getFrequencyOf(tests[index]));
        }
    } // end testFrequency

    // Tests the method contains.
    private static void testContains(BagInterface<String> aBag, String[] tests) {
        System.out.println("\nTesting the method contains:");
        for (int index = 0; index < tests.length; index++) {
            System.out.println("Does this bag contain " + tests[index]
                    + "? " + aBag.contains(tests[index]));
        }
    } // end testContains

    // Tests the method toArray while displaying the bag.
    private static void displayBag(BagInterface<String> aBag) {
        System.out.println("The bag contains " + aBag.getCurrentSize()
                + " string(s), as follows:");
        Object[] bagArray = aBag.toArray();
        for (int index = 0; index < bagArray.length; index++) {
            System.out.print(bagArray[index] + " ");
        } // end for

        System.out.println();
    } // end displayBag
} // end LinkedBagDemo




 Creating an empty bag.
 The bag contains 0 string(s), as follows:

```

```
Testing isEmpty with an empty bag:
isEmpty finds the bag empty: OK.

Testing the method getFrequencyOf:
In this bag, the count of  is 0
In this bag, the count of B is 0

Testing the method contains:
Does this bag contain ? false
Does this bag contain B? false

Removing a string from the bag:
remove() returns null
The bag contains 0 string(s), as follows:

Removing "B" from the bag:
remove("B") returns false
The bag contains 0 string(s), as follows:

Adding to the bag: A D B A C A D
The bag contains 7 string(s), as follows: D A C A B D A
Testing isEmpty with a bag that is not empty:
isEmpty finds the bag not empty: OK.

Testing the method getFrequencyOf:
In this bag, the count of A is 3
In this bag, the count of B is 1
In this bag, the count of C is 1
In this bag, the count of D is 2
In this bag, the count of Z is 0

Testing the method contains:
Does this bag contain A? true
Does this bag contain B? true
Does this bag contain C? true
Does this bag contain D? true
Does this bag contain Z? false

Removing a string from the bag:
remove() returns D
The bag contains 6 string(s), as follows: A C A B D A

Removing "B" from the bag:
remove("B") returns true
The bag contains 5 string(s), as follows: C A A D A

Removing "A" from the bag:
remove("A") returns true
The bag contains 4 string(s), as follows: C A D A

Removing "C" from the bag:
remove("C") returns true
The bag contains 3 string(s), as follows: A D A

Removing "Z" from the bag:
remove("Z") returns false
The bag contains 3 string(s), as follows: A D A

Clearing the bag:
Testing isEmpty with an empty bag:
isEmpty finds the bag empty: OK.

The bag contains 0 string(s), as follows:
```